

Concepts of Programming Languages, Spring Term 2026

Project - Task 1: Prolog

Intermediate: 13th April 2026 23:59 - Final: 15th May 2026 23:59

1. Project Description

In this project, you are going to implement some utilities for a restaurant that help it manage reservations and supplies.

2. Knowledge Base You are given a knowledge base containing the following predicates

- `group(Name, Count, Time).`
- `staff(Day, Count)`
- `tables(Tables)`
- `recipe(DishName, [Ingredients])`
- `order(GroupName, DishNames)`

The predicates and their arguments are described in comments in the sample knowledge base provided. You are provided with two knowledge bases:

- `sample_KB.pl` which contains a very small KB for basic testing and it has the fact upon which examples in this description are based.
- `public_KB.pl` which has a more extensive set of tests and this will be part of the grading.

To use a knowledge base to test your code, add the following line at the top of your code file: `:-consult('KBname.pl')` replacing KBname with the name of the knowledge base you currently want to use. You can only consult one KB at a time to avoid conflicting results. The `tests.xlsx` file provided included two tabs named public and sample which include test queries, expected outputs and explanation of the output in some cases. You are allowed (and encouraged) to come up with your own additional KBs and test cases.

3. Implementation Requirements Below are the predicates you are required to implement along with their success conditions and their argument formats. You will need to implement additional predicates to break down the logic over them for a more organised structure.

a) `check_staff(Day, Time, Reservations)`

- **Day:** A date structure in the form `day(D, M)`
- **Time:** A constant either `morning` or `evening`
- **Reservations:** A list of reservations where each reservation has the form:
`res(Day, Time, GroupName, TableName)`

This predicate succeeds when all the **Reservations** do not violate the constraint on staff counts for the given day and time. At each timing, there must be no more tables reserved than staff available on that day.

Examples:

Query 1:

```
?- check_staff(day(15, 2), morning, [res(day(15, 2), morning, a, t1),
                                     res(day(15, 2), morning, c, t2)]).
```

Output 1: false.

Query 2:

```
?- check_staff(day(15, 2), morning, [res(day(15, 2), morning, a, t1),
                                     res(day(15, 2), evening, b, t2), res(day(15, 2), evening, d, t1)]).
```

Output 2: true.

b) `schedule_all_reservations(Days, Schedule)`

- **Days:** A list of available dates in the form `[day(D1, M1), day(D2, M2), ...]`
- **Schedule:** A list of reservations in the same format as in part a.

This predicate succeeds when **Schedule** is a valid reservation schedule for all the groups in the KB across the **Days** given. A schedule is valid if:

- Each group is assigned to exactly one table with a capacity greater than or equal to the group size.
- Each group is assigned on the time it prefers.
- No two groups are assigned the same table at the same time on the same day.
- For each day and time, the number of tables filled cannot exceed the number of staff members available on that day.

Examples:

Query 1: `?- schedule_all_reservations([day(15, 2), day(17, 2)], R).`

Output 1:

```
R = [res(day(15, 2), morning, a, t1), res(day(15, 2), evening, b, t2),
      res(day(17, 2), morning, c, t2), res(day(17, 2), evening, d, t1)] ;
R = [res(day(15, 2), evening, b, t2), res(day(15, 2), morning, a, t1),
      res(day(17, 2), evening, d, t1), res(day(17, 2), morning, c, t2)] ;
...
```

Query 2: `schedule_all_reservations([day(15, 2)], R).`

Output 2: false.

c) `group_ingredients(GroupName, Ingredients)`

- **GroupName:** A single group's name
- **Ingredients:** A list of ingredients.

`group_ingredients` succeeds when **Ingredients** is the list of all ingredients required by the **GroupName** for all the dishes they are ordering. An ingredient that is required multiple times by multiple dishes should be repeated as many times as it is required. Each ingredient is required by each dish only once. **Examples:**

Query 1: `?- group_ingredients(b, I).`

Output 1: `I = [ing5, ing6, ing5, ing6, ing5, ing6, ing5, ing6].`

Query 2: `?- group_ingredients(b, [ing5, ing6]).`

Output 2: false.

d) `needed_ingredients(Reservations, AllIngredients)`

- **Reservations:** A list of reservations in the form:
`[reservation(Day1, Time1, Group1, Table1), ...]`.
- **AllIngredients:** A list of (Date, IngredientList) pairs grouped by date (not necessarily sorted) in the form :
`[(day(D1, M1), [Ingredient1, Ingredient2, ...]), (day(D2, M2), [...]), ...]`

`needed_ingredients` succeeds when `AllIngredients` provides the list of ingredients needed to cover the orders of each all groups in `Reservations`. An ingredient that is needed for multiple orders should be repeated that many times in the list. Ingredients should be grouped by day (a day is not repeated).

Examples:

Query 1:

```
?- needed_ingredients([res(day(15, 2), morning, a, t1),
    res(day(15, 2), evening, b, t2), res(day(17, 2), morning, c, t2),
    res(day(17, 2), evening, d, t1)], I).
```

Output 1:

```
I = [(day(15, 2), [ing1, ing2, ing3,
    ing5, ing6, ing5, ing6, ing5, ing6, ing5, ing6]),
    (day(17, 2), [ing1, ing3, ing4, ing5, ing1, ing2, ing3,
    ing5, ing6, ing1, ing2, ing3, ing1, ing3, ing4, ing5])].
```

Query 2:

```
?- needed_ingredients([res(day(15, 2), morning, a, t1),
    res(day(15, 2), evening, b, t2), res(day(17, 2), morning, c, t2),
    res(day(17, 2), evening, d, t1)],
    [(day(15, 2), [ing1, ing2, ing3]) ,
    (day(15, 2), [ing5, ing6, ing5, ing6, ing5, ing6, ing5, ing6]),
    (day(17, 2), [ing1, ing3, ing4, ing5, ing1, ing2, ing3]) ,
    (day(17, 2), [ing5, ing6, ing1, ing2, ing3, ing1, ing3, ing4, ing5])]).
```

Output 2: `false`.

- e) `write_reservations_to_csv(Filename, Schedule)` This predicate writes all the reservations in schedule to a `.csv` file with columns: Day,Month,Time,Group,Table in that order. Examples of valid output files are provided in the starter folder in the CMS as `sample_reservations.csv` and `public_reservations.csv` for the sample and public KBs respectively.
- f) `write_ingredients_to_csv(Filename, AllIngredients)` This predicate writes the list of ingredients for each day in a row in a `.csv` file with the columns: Day,Month,Ingredients in that order where Ingredients is the list of all ingredients for that day separated by semicolons. An example of the output is provided in the starter folder in the CMS as `sample_shoppinglist.csv` and `public_shoppinglist.csv` for the sample and public KBs respectively.

4. Testing notes: In some cases, a query might have multiple possible outputs but in the description and test files, we only show a few outputs for readability. If a query has a list as an argument, the order of elements in that list should not affect whether the query succeeds or not unless otherwise specified.